Web 2 – Séance 06

Gestion d’événements

Événements dans le navigateur

- Un **événement** est une action qui se produit dans le navigateur
- Exemples : clic de souris, frappe clavier, soumission d'un formulaire, chargement d'une image, etc.
- Le navigateur **déclenche** un événement et on peut **écouter** cet événement avec JavaScript
- Quand l'événement se produit, le navigateur appelle la fonction qu'on a définie pour cet événement (le **callback**)

Événements courants

- `click` : clic de souris
- `submit` : envoi d'un formulaire
- `input` : changement dans un input (au fur et à mesure)
- `change` : valeur validée (perte de focus ou Entrée pour un champ texte, immédiat pour select/checkbox)
- `keydown` : touche enfoncée
- `keyup` : touche relâchée
- `mouseover` , `mouseout` : souris entre/sort d'un élément
- `focus` , `blur` : input reçoit/perd le focus

Fonction `addEventListener`

```html
<button id="my-button">Cliquez-moi</button>
```

```js
const button = document.querySelector('#my-button');

button.addEventListener('click', function(event) {
  console.log('Bouton cliqué!');
});

// Ou avec une arrow function
button.addEventListener('click', (event) => {
  console.log('Bouton cliqué!');
});
```

L’objet Event

```html
<div id="outer">
  <button id="my-button">Cliquez-moi</button>
</div>
```

```js
const outer = document.querySelector('#outer');

outer.addEventListener('click', (event) => {
  // event.target: l'élément qui a déclenché l'événement (celui qui a été cliqué)
  console.log(event.target.id); // "my-button"

  // event.currentTarget: l'élément sur lequel on a mis le listener
  console.log(event.currentTarget.id); // "outer"

  // Autres propriétés
  console.log(event.type); // "click"
  console.log(event.timeStamp); // millisecondes depuis le chargement de la page
});
```

```js
document.addEventListener('keydown', (event) => {
  console.log('Touche enfoncée:', event.key); // "a", "Enter", "ArrowUp", etc.
  console.log('Code de la touche:', event.code); // "KeyA", "Enter", "ArrowUp", etc.
});
```

event.preventDefault()

Empêcher le comportement par défaut.

Comportement par défaut d'un formulaire : recharger la page.

```html
<form id="my-form">
  <input type="text" name="name" placeholder="Nom">
  <button type="submit">Envoyer</button>
</form>
```

```javascript
// Exemple 1: Empêcher la soumission du formulaire
const form = document.querySelector('form');

form.addEventListener('submit', (event) => {
  event.preventDefault(); // Ne recharge pas la page!

  const name = document.querySelector('input[name="name"]').value;
  console.log('Recette ajoutée:', name);
});
```

event.preventDefault() (suite)

Empêcher le comportement par défaut.

Comportement par défaut d'un lien : naviguer vers l'URL.

```
<a href="https://www.google.com" id="my-link">Aller sur Google</a>
```

```
// Exemple 2: Empêcher un lien de naviguer
const link = document.querySelector('a');

link.addEventListener('click', (event) => {
  event.preventDefault();
  console.log('Navigation bloquée');
});
```

Autres comportements par défaut : clic droit, drag & drop, etc.

Event Bubbling et Propagation

Les événements "remontent" l'arbre du DOM!

```
<div id="parent">
  <button id="child">Cliquer</button>
</div>
```

```
const parent = document.querySelector('#parent');
const child = document.querySelector('#child');

child.addEventListener('click', () ⇒ console.log('Événement sur le bouton'));

parent.addEventListener('click', () ⇒ console.log('Événement sur le parent (bubbling)'));

// Output si on clique sur le bouton:
// "Événement sur le bouton"
// "Événement sur le parent (bubbling)"
```

Arrêter la propagation:

```
child.addEventListener('click', (event) ⇒ {
  event.stopPropagation(); // N'ira pas au parent
});
```

Event Delegation: Écouter un parent

```html
<div id="recipes-list">
  <button class="recipe-btn" data-id="1">Carbonara</button>
  <button class="recipe-btn" data-id="2">Risotto</button>
  <button class="recipe-btn" data-id="3">Soupe</button>
</div>
```

```javascript
const list = document.querySelector('#recipes-list');

// Un seul listener sur le parent!
list.addEventListener('click', (event) => {
  // Vérifier si c'est un bouton qu'on veut
  if (event.target.classList.contains('recipe-btn')) {
    const recipeId = event.target.dataset.id;
    console.log('Recette cliquée:', recipeId);
  }
});
```

Avantages :

- Moins de mémoire
- Fonctionne aussi pour les éléments ajoutés dynamiquement!

Exemple: Basculer "Favori" sur une carte

```html
<div class="recipe-card" data-id="42">
  <h2>Pâtes Carbonara</h2>
  <button class="favorite-btn">🤍 Ajouter aux favoris</button>
</div>
```

```javascript
const cards = document.querySelectorAll('.recipe-card');

cards.forEach(card => {
  const btn = card.querySelector('.favorite-btn');

  btn.addEventListener('click', () => {
    // Basculer la classe
    card.classList.toggle('favorite');

    // Changer le texte du bouton
    if (card.classList.contains('favorite')) {
      btn.textContent = '❤️ Retirer des favoris';
      btn.style.color = 'red';
    } else {
      btn.textContent = '🤍 Ajouter aux favoris';
      btn.style.color = 'gray';
    }
  });
});
```

Exemple: Recherche en direct

```html
<input id="search" type="text" placeholder="Rechercher une recette ... ">
<div id="recipes-list"></div>
```

```javascript
const recipes = [{ id: 1, title: 'Carbonara' }, { id: 2, title: 'Risotto' }, { id: 3, title: 'Soupe' }];

const searchInput = document.querySelector('#search');
const listContainer = document.querySelector('#recipes-list');

function renderRecipes(filter = '') {
  listContainer.innerHTML = '';

  const filtered = recipes.filter(r => r.title.toLowerCase().includes(filter.toLowerCase()));
  filtered.forEach(recipe => {
    const div = document.createElement('div');
    div.textContent = recipe.title;
    listContainer.appendChild(div);
  });
}

// Au démarrage
renderRecipes();

// À chaque frappe
searchInput.addEventListener('input', (event) => renderRecipes(event.target.value));
```

Récapitulatif

Concept	Code
Ajouter un listener	`addEventListener(event, callback)`
Accéder à l'élément	`event.target`
Empêcher le défaut	`event.preventDefault()`
Arrêter la propagation	`event.stopPropagation()`
Sélectionner plusieurs	`querySelectorAll()` + `forEach()`
Event delegation	Listener sur parent + check `target`

Prochaine séance : Séance 07 — Projet Vite et composant React 🚀

Exercice complémentaire EC02

1. Téléchargez la page HTML de la séance 06 sur moodle et placez-la dans votre dossier de cours
2. Créez un fichier `script.js` et liez-le à la page HTML
3. Quand on clique sur le bouton "Ajouter une recette", un formulaire apparaît dans le conteneur `#add-recipe`, avec les champs "Titre" et "Durée" et un bouton de soumission
4. Quand on soumet le formulaire, la recette est ajoutée à la liste du conteneur `#recipes-list` (même structure que les cartes existantes) et le formulaire disparaît de `#add-recipe`
5. **Optionnel** : le bouton "Supprimer" de chaque carte retire la carte de la liste – y compris pour les recettes ajoutées après le chargement de la page (pensez à l'event delegation)
6. **Optionnel** : le champ `#search` filtre les cartes affichées à chaque frappe (masquer les cartes dont le titre ne contient pas le texte recherché)